

Методология проекта Adult Census Income

Проект: предсказание, превышает ли годовой доход человека 50 000 долларов, по признакам из Adult/Census Income dataset. Формат задачи - бинарная классификация на табличных данных.

Документ написан как design document для студенческого проекта: не только теория, но и обоснование выбора методов, инструментов и ограничений.

1. Что именно мы решаем

Целевая переменная принимает два значения: $\leq 50K$ и $> 50K$. В ML-формулировке это supervised learning: для каждого объекта есть признаки X и известная метка y , а модель должна научиться предсказывать y для новых объектов.

Положительным классом считаем $> 50K$. Это важно для метрик: precision, recall и F1 считаем именно для класса $> 50K$, потому что он менее частый и более интересный для анализа.

2. Почему выбран Adult Census Income

Adult подходит под требования проекта лучше, чем слишком простые игрушечные датасеты. Он известный, но не тривиальный: есть числовые и категориальные признаки, пропуски, дисбаланс классов, разные типы признаков и этические ограничения.

- Есть preprocessing: символ ? нужно обрабатывать как пропуск; категориальные признаки нельзя напрямую подать в большинство моделей sklearn; числовые признаки нужно масштабировать для линейной модели и kNN.
- Стартовая accuracy может выглядеть неплохо из-за большинства класса $\leq 50K$, поэтому одной accuracy недостаточно. Это заставляет использовать более осмысленные метрики.
- Данные достаточно большие для учебного проекта: 48 842 объекта и 14 признаков в полном наборе UCI. Это не игрушечные 150 строк, но и не неподъемный датасет.
- Тема понятная на защите: предсказание доходной категории по социально-демографическим и рабочим признакам.

3. Стек курса и что из него используем

По репозиторию курса стек включает Python, numpy, matplotlib, pandas, preprocessing, scikit-learn, feature engineering, encoding категориальных признаков, variance threshold, линейные модели, logistic regression, decision trees, kNN, random forest и boosting. В requirements указаны jupyter, ipykernel, matplotlib, numpy, pandas, scikit-learn, seaborn, tqdm.

Поэтому в проекте используем:

- pandas/numpy - загрузка, очистка, преобразование таблиц, простая feature engineering;
- matplotlib/seaborn - графики EDA;
- scikit-learn Pipeline и ColumnTransformer - воспроизводимый preprocessing без leakage;
- SimpleImputer - заполнение пропусков;
- OneHotEncoder - кодирование категориальных признаков;
- StandardScaler - масштабирование числовых признаков для LogisticRegression и kNN;
- VarianceThreshold - простой feature selection, пройденный в курсе;
- LogisticRegression, DecisionTreeClassifier, KNeighborsClassifier, RandomForestClassifier, AdaBoostClassifier/GradientBoostingClassifier - модели в рамках классического ML;
- GridSearchCV/StratifiedKFold - аккуратная, но небольшая настройка гиперпараметров.

XGBoost/CatBoost/LightGBM в курсе упоминаются на практикуме по бустингам, но в основной версии проекта мы не делаем их обязательными. Причина практическая: они не входят в

минимальный requirements репозитория, а проект должен запускаться у проверяющего без дополнительной настройки. XGBoost можно оставить optional-блоком.

4. Методология

4.1. Общая схема

- Загружаем train/test из UCI.
- Проводим EDA: типы признаков, пропуски, распределение target, числовые распределения, зависимость target от образования, занятости, часов работы.
- Создаем несколько простых новых признаков: net capital, log capital gain/loss, флаги наличия gain/loss, группы возраста и рабочих часов.
- Делим обучающие данные на train/validation с сохранением долей классов.
- Собираем единый preprocessing pipeline.
- Сравниваем несколько моделей на validation.
- Для 1-2 лучших моделей делаем небольшой GridSearchCV.
- Финальную модель переобучаем на полном train и оцениваем на official test.
- Делаем error analysis и анализ ограничений.

4.2. Почему Pipeline

Pipeline нужен, чтобы избежать data leakage. Например, imputer и scaler должны обучаться только на train, а потом применяться к validation/test. Если сначала масштабировать весь датасет, тестовая информация попадет в обучение. Pipeline делает процесс аккуратным и воспроизводимым.

4.3. Почему ColumnTransformer

В Adult смешаны числовые и категориальные признаки. Для них нужны разные преобразования. ColumnTransformer позволяет описать это в одном объекте: numeric pipeline и categorical pipeline.

5. Метрики

Assurasy оставляем, но не считаем главной метрикой. В Adult большинство объектов относится к $\leq 50K$, поэтому DummyClassifier, который всегда предсказывает majority class, уже даст заметную ассурасу.

- ассурасу - общая доля правильных ответов, нужна как привычная метрика, но может быть обманчивой;
- balanced_ассурасу - среднее качество по классам, лучше показывает работу при дисбалансе;
- precision для $>50K$ - какая доля предсказанных $>50K$ действительно $>50K$;
- recall для $>50K$ - какую долю настоящих $>50K$ модель нашла;
- F1 для $>50K$ - компромисс precision и recall, используем для выбора модели;
- ROC-AUC - качество ранжирования по вероятности класса $>50K$.

6. Почему выбраны именно эти модели

Модель	Зачем берем	Ожидаемые плюсы/минусы
DummyClassifier	Baseline	Показывает, насколько задача сложнее majority prediction.
LogisticRegression	Линейная интерпретируемая модель	Быстрая, понятная; может недоучить нелинейности.
DecisionTree	Базовая нелинейная модель	Хорошо объясняется; склонна к переобучению.
KNN	Метод из курса для сравнения	Простая идея; на one-hot и больших данных может быть медленным.
RandomForest	Сильный классический baseline	Устойчивее дерева; хуже интерпретируется, чем одно дерево.
AdaBoost/GradientBoosting	Бустинг из курса	Часто силен на табличных данных; нужно контролировать переобучение.

7. Почему не выбираем другие методы

- Нейросети: запрещены правилами проекта без approval; для такого табличного датасета они не нужны и сложнее защищаются.
- SVM как основная модель: теоретически можно, но на 48k строк и one-hot признаках может быть медленно; в README курса SVM не является центральной темой.
- Naïve Bayes: простая модель, но она не указана в основной траектории курса; кроме того, предположение независимости признаков для Adult сомнительно.
- CatBoost как главная модель: хорош для категориальных признаков, но скрывает часть preprocessing, который по требованиям проекта как раз нужно показать. Можно упомянуть как optional.
- LightGBM/XGBoost как обязательная модель: бустинги упоминались, но эти библиотеки не входят в минимальный requirements. Чтобы проект был воспроизводимым, обязательная часть остается на sklearn.
- Сложные fairness/debiasing методы: тема чувствительная, но специальные fairness-алгоритмы выходят за рамки курса. Вместо этого делаем честное ограничение и простую проверку варианта без sex/race.

8. Validation design

Используем official train/test split из UCI. Внутри train дополнительно выделяем validation через train_test_split со stratify. Это позволяет выбирать модель без подсматривания в official test.

- Validation split: выбор модели и предварительная настройка.
- Cross-validation: небольшой GridSearchCV для лучших моделей.
- Official test: финальная оценка один раз после выбора модели.

9. Error analysis

После финальной оценки смотрим confusion matrix и разбираем false positives/false negatives. Особенно полезно смотреть, в каких группах модель чаще ошибается: education, marital_status, occupation, sex, race. Это не доказывает причинность, но помогает описать ограничения.

10. Риски и ограничения

- Данные старые: они основаны на переписи 1994 года, поэтому модель не отражает современный рынок труда.
- Признаки sex и race чувствительные. Нельзя использовать такую модель для реальных решений о людях без отдельной экспертизы.

- Порог 50K исторический и сейчас не эквивалентен тому же уровню дохода из-за инфляции.
- Модель учит корреляции, а не причинно-следственные зависимости.
- Некоторые признаки, например education и education_num, частично дублируют информацию; это нужно учитывать при интерпретации.

11. Минимальный план защиты

- Показать, что задача не тривиальная: пропуски, категориальные признаки, дисбаланс классов.
- Показать baseline и объяснить, почему accuracy недостаточно.
- Показать единый Pipeline и ColumnTransformer.
- Сравнить несколько моделей из курса.
- Обосновать финальный выбор по F1/ROC-AUC/balanced accuracy.
- Показать confusion matrix, ROC/PR curves и 2-3 вывода из error analysis.
- Честно сказать про ограничения и чувствительные признаки.

12. Источники

1. Репозиторий курса: <https://github.com/nuvard/spbu-ai-fundamentals>
2. UCI Adult / Census Income dataset: <https://archive.ics.uci.edu/dataset/2/adult>
3. UCI raw data files: <https://archive.ics.uci.edu/ml/machine-learning-databases/adult/>
4. scikit-learn documentation: <https://scikit-learn.org/stable/>